

Engineering Decisions & Thought Log (Final Build Strategy)

The "Dual-Environment" Philosophy

I'm not just building an app; I'm building a resilient data platform. By creating two environments —**VistaStream (Enterprise)** and **NeonPlay (Startup)**—I can prove that the system isn't just a toy that works on clean data.

- **VistaStream:** Clean, standardized, mature governance. This shows the system's ability to provide high-level analytics for a polished organization.
- **NeonPlay:** Messy, inconsistent, and rushed. This shows the system's ability to handle "real-world" operational noise (e.g., "APAC" vs "Asia Pacific") without breaking.

Ingestion & Observability (The Ingestion Logs)

I've added an `ingestion_logs` table.

- **Choice C (Log & Reject):** For the startup dataset, when I hit duplicate Primary Keys or critical malformed data, I'm logging it and rejecting the row rather than silently fixing it. In a Quant environment, silent data fixes are dangerous. This log serves as an "Audit Trail" for management to see exactly how "noisy" their data actually is.

Architecture: Tiered Access & Masking

- **The "Golden Rule":** The LLM is a "Masked Entity." It never sees the full database or raw PDF files. It only gets specific snippets via tool-calling.
- **Traceability:** The "Direct Fetch" path is load-bearing. The UI gets a Row ID or Snippet ID from the LLM, but it fetches the actual "Ground Truth" directly from the source. This prevents the LLM from being a bottleneck for truth and ensures no hallucinations in the "Trace View."

Logic Layer: Scratch-built Orchestration

I am avoiding LangChain. I want to show that I can manage the state machine and the tool-routing logic myself. It proves I understand how the prompt-response-tool loop actually operates under the hood.

The SQL "Middle Ground"

- **Approach:** Validated Text-to-SQL.

- **Mechanism:** The LLM generates SQL, but a Python-based "Security Guard" parses it to ensure no destructive commands (DROP, DELETE) are present and that it adheres to the schema map I provide. This gives the flexibility of raw SQL with the safety of a sandbox.

UI Decisions

- **Persona:** Management-focused (White Mode).
- **Debouncing:** Every button is debounced. Management users are chaotic; the UI shouldn't allow them to trigger 5 API calls by spamming a button.
- **Trace View:** Highly granular. I'm aiming for sentence-level highlighting in the PDFs and